

T.Sai Charan
Dept of CSE (Networks)
Kakatiya Institute of Technology and
Science
Warangal, India
b22cn046@kitsw.ac.in

D.Uma Mahesh
Dept of CSE (Networks)
Kakatiya Institute of Technology and
Science
Warangal, India
b22cn020@kitsw.ac.in

K.Manideep
Dept of CSE (Networks)
Kakatiya Institute of Technology and
Science
Warangal, India
b22cn036@kitsw.ac.in

B.Narendar
Dept of CSE (Networks)
Kakatiya Institute of Technology and
Science
Warangal, India
b22cn133l@kitsw.ac.in

P.Nikil
Dept of CSE (Networks)
Kakatiya Institute of Technology and
Science
Warangal, India
b21cn004@kitsw.ac.in

V.Abhinay
Dept of CSE (Networks)
Kakatiya Institute of Technology and
Science
Warangal, India
lb21cn050@kitsw.ac.in

Abstract—Cloud computing has become an essential technology for storing and sharing large amounts of data across distributed environments. However, protecting sensitive information during storage and processing remains a major challenge in multi-cloud systems. This project presents a privacy preserving data sharing framework using Homomorphic Encryption (HE) to secure user data while allowing encrypted computation on multiple cloud servers. The proposed system is implemented using the CKKS scheme provided by the TenSEAL library and integrates Flask-based cloud APIs, Render cloud deployment, and Replit R

either enter numerical values manually or upload numerical datasets through an interactive web interface. Data is encrypted at the client side before being transmitted to the cloud, ensuring that cloud providers never access plaintext information. Cloud servers perform operations such as encrypted sum and average computation directly on ciphertexts, and the final result is decrypted only by the client. The frontend provides separate modules for numerical input and dataset processing, making the system user friendly and suitable for healthcare, finance, and research applications. Experimental results demonstrate that the proposed system provides secure computation, data confidentiality, and privacy preservation while maintaining practical usability in multi-cloud environments.

Keywords— Homomorphic Encryption, Multi-Cloud Computing, Privacy Preservation, TenSEAL, CKKS Scheme, Data Security, Flask API, Cloud Computing, Dataset Encryption, Secure Data Sharing.

I. INTRODUCTION

The rapid growth of cloud computing technologies has enabled organizations and individuals to store, process, and share large volumes of data over distributed cloud infrastructures. Multi-cloud environments provide scalability, reliability, and cost efficiency by distributing services across multiple cloud providers. However, data privacy and confidentiality remain significant concerns because sensitive information stored on cloud servers may be exposed to unauthorized access or malicious attacks. Traditional encryption techniques protect data during storage and transmission but require decryption before processing, which introduces security risks.

Homomorphic Encryption (HE) overcomes this limitation by allowing computations to be performed directly on encrypted data without revealing the original plaintext. In this project, a privacy preserving multi-cloud architecture is implemented using the CKKS homomorphic encryption scheme with the TenSEAL library. The system consists of a client-side encryption module, cloud APIs deployed on Render and Replit, and a modern frontend interface supporting both numerical inputs and dataset uploads. The proposed solution demonstrates secure encrypted computation while ensuring that user

data remains confidential throughout the process.

II. LITERATURE REVIEW

Existing cloud security solutions mainly focus on protecting stored data using standard encryption methods such as AES and RSA. Although these methods secure data at rest and during transmission, they require decryption before computation, exposing sensitive information to cloud providers. Researchers introduced Homomorphic Encryption to solve this issue by enabling computation directly on encrypted data.

Several studies demonstrate the use of partially and fully homomorphic encryption schemes in healthcare, finance, and cloud analytics applications. The CKKS scheme is widely preferred for approximate arithmetic operations on real numbers because it supports efficient encrypted computation with acceptable performance. Libraries such as Microsoft SEAL and TenSEAL simplified the practical implementation of homomorphic encryption in real-world systems. Recent research also emphasizes multi-cloud security architectures to avoid dependency on a single provider and improve availability and resilience. The proposed project extends these concepts by integrating cloud APIs, encrypted dataset processing, and a user-friendly frontend for secure data sharing.

III. SYSTEM DESIGN AND IMPLEMENTATION

The proposed system is divided into three main modules: frontend interface, client-side encryption module, and multi-cloud

processing servers. The frontend is developed using HTML, CSS, and JavaScript and provides two separate options: numerical value input and dataset upload. Users can enter values manually or upload CSV datasets containing numerical data.

The client-side module uses the TenSEAL library to generate CKKS encryption contexts and encrypt the selected data before transmission. Encrypted ciphertexts and public contexts are sent to cloud servers through Flask APIs. Cloud servers deployed on Render and Replit receive ciphertexts and perform encrypted operations such as sum and average computation without decrypting the data. The resulting encrypted ciphertext is returned to the client, where decryption is performed using the secret key.

The system architecture ensures that sensitive data remains protected during storage, transmission, and processing. Since cloud servers only operate on ciphertexts, privacy preservation is achieved throughout the computation lifecycle.

IV. RESULTS AND DISCUSSION

The implementation successfully demonstrates secure encrypted computation in a multi-cloud environment. Users are able to input numerical values and upload datasets through the graphical interface. The frontend displays encrypted ciphertext samples, encrypted computation results, and final decrypted outputs. Experimental testing shows that cloud servers can perform arithmetic operations directly on

encrypted data without accessing the original plaintext.

The system was tested using healthcare datasets and numerical sample data. Operations such as encrypted sum and average calculations were successfully executed across cloud platforms. The results confirm that Homomorphic Encryption can provide strong privacy guarantees while supporting practical cloud computation. The integration of multiple cloud servers also improves reliability and demonstrates the feasibility of secure distributed processing.

V. CONCLUSION

This project presents a privacy preserving data sharing system using Homomorphic Encryption in a multi-cloud environment. The proposed architecture enables secure encrypted computation without exposing sensitive user data to cloud providers. By integrating TenSEAL, Flask APIs, Render cloud deployment, and Replit services, the system demonstrates secure and practical encrypted data processing.

The developed frontend supports both numerical input and dataset upload, improving usability and making the project suitable for real-world applications in healthcare, finance, and research domains. Experimental results verify that encrypted operations such as sum and average computation can be performed securely on cloud servers. Future work may include implementing advanced encrypted machine learning models, larger dataset processing, and improved cloud scalability.

REFERENCES

- [1] C. Gentry, 'Fully Homomorphic Encryption Using Ideal Lattices,' STOC, 2009.
- [2] Microsoft SEAL Homomorphic Encryption Library Documentation.
- [3] TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption.
- [4] P. Paillier, 'Public-Key Cryptosystems Based on Composite Degree Residuosity Classes,' EUROCRYPT, 1999.
- [5] Research Papers on Privacy Preserving Cloud Computing and Multi-Cloud Security.